K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Lab Assignment-4

School of Engineering and Technology

Name = Tamanna

RollNo. = 2401201138

Program =BCA (AI & DS)

Fundamental of Operating System

Lab ENCA252

Submitted to :- Jyoti Yadav

Implementation and Analysis of Disk Scheduling Algorithms

Problem Statement

Disk scheduling is an essential function of operating systems that determines the order in which disk I/O requests are serviced. Efficient disk scheduling improves system performance by reducing seek time and increasing throughput.

In this assignment, students will implement and analyze various disk scheduling algorithms such as **FCFS, SSTF, SCAN, and C-SCAN**. The goal is to simulate head movements and calculate total seek time

Learning Objectives

- Understand disk scheduling concepts
- Implement various disk scheduling algorithms
- Calculate total head movement (seek time)
- Compare performance of different algorithms

Tools/Technology Used

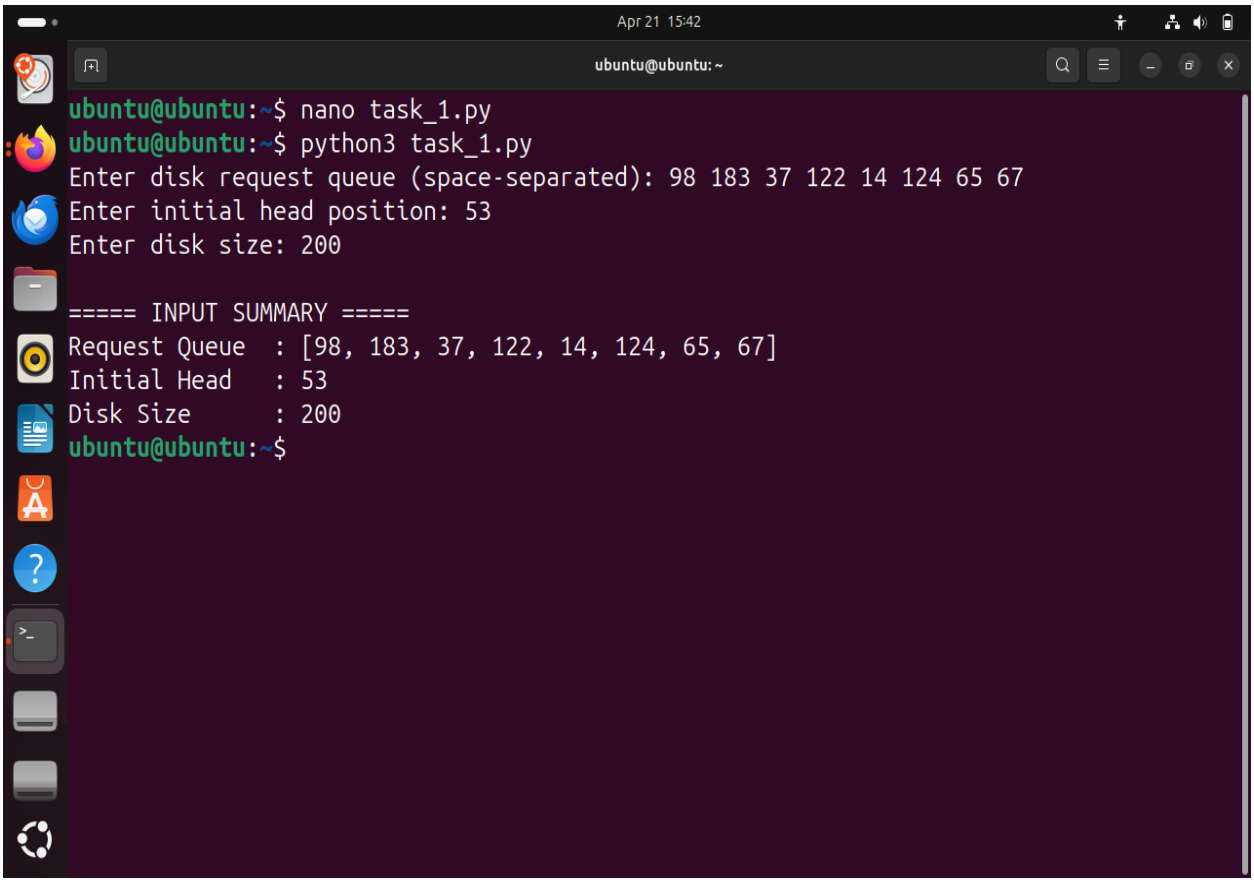
- Python 3.x
- Basic Python libraries
- Linux/ Ubuntu

Task 1: Input and Disk Request Representation

Code :-

```
requests = list(map(int, input("Enter disk requests: ").split()))
head = int(input("Enter head position: "))
disk_size = int(input("Enter disk size: "))
print(f"\nRequests  : {requests}")
print(f"Head      : {head}")
print(f"Disk Size : {disk_size}")
```

Output:-

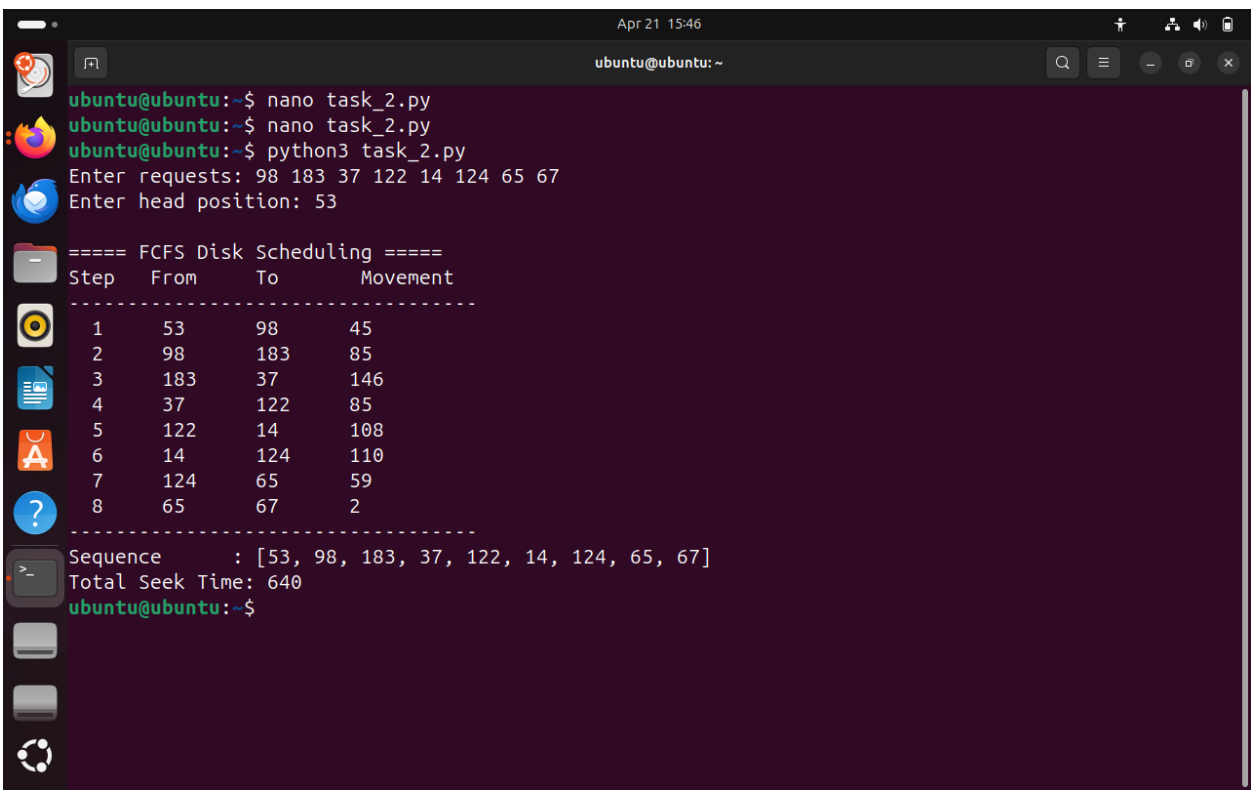


Task 2: FCFS Disk Scheduling

Code :-

```
requests = list(map(int, input("Enter requests: ").split()))
head = int(input("Enter head position: "))
seek_time = 0
print("\n===== FCFS =====")
for req in requests:
    move = abs(head - req)
    print(f"{head} -> {req} = {move}")
    seek_time += move
    head = req
print(f"Total Seek Time: {seek_time}")
```

Output:-

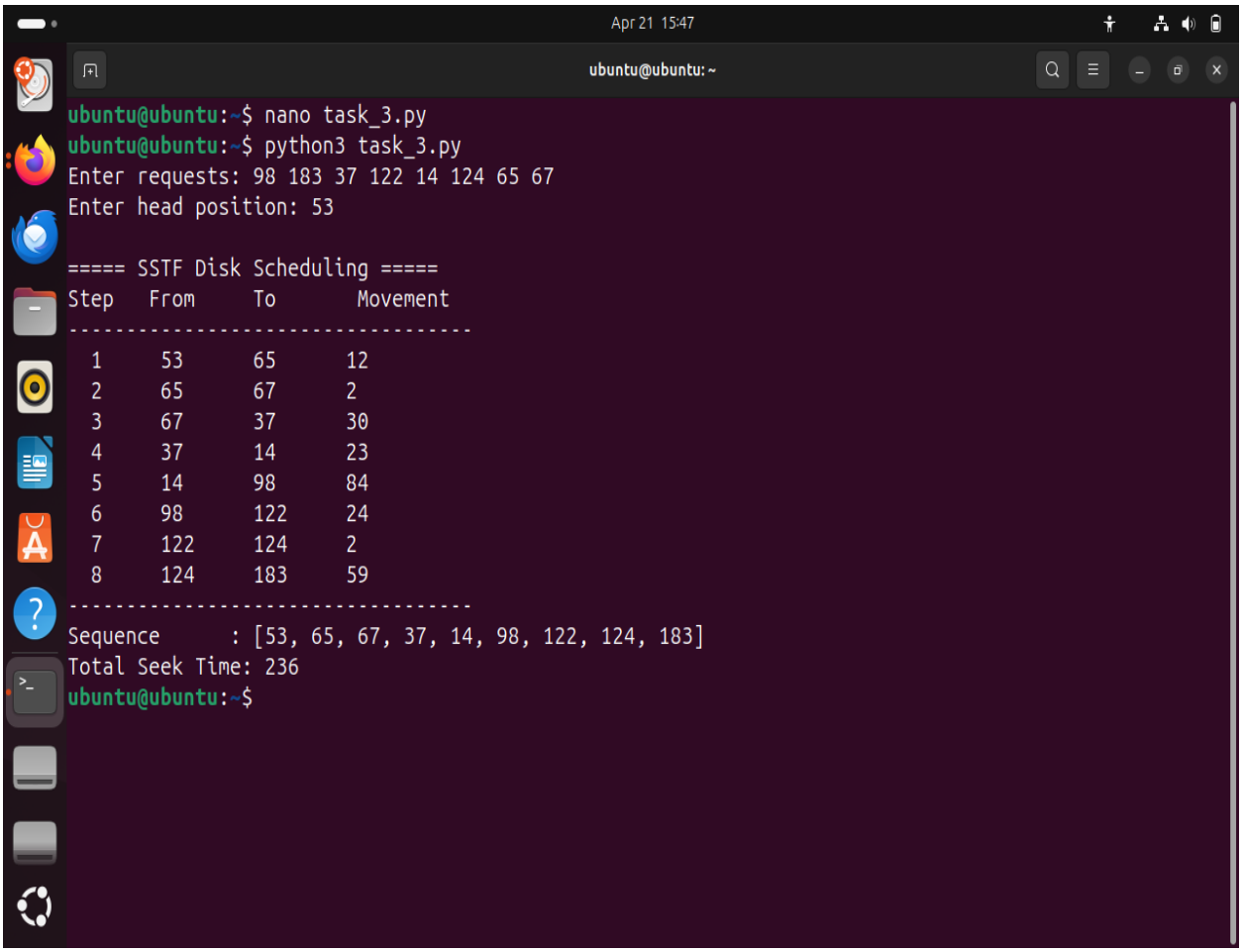


Task 3: SSTF (Shortest Seek Time First)

Code :-

```
requests = list(map(int, input("Enter requests: ").split()))
head = int(input("Enter head position: "))

seek_time, remaining = 0, requests.copy()
print("\n===== SSTF =====")

while remaining:
    nearest = min(remaining, key=lambda x: abs(x - head))
    move = abs(head - nearest)
    print(f"{head} -> {nearest} = {move}")
    seek_time += move
    head = nearest
    remaining.remove(nearest)
print(f"Total Seek Time: {seek_time}")
```

Output:-



```
ubuntu@ubuntu:~$ nano task_3.py
ubuntu@ubuntu:~$ python3 task_3.py
Enter requests: 98 183 37 122 14 124 65 67
Enter head position: 53

===== SSTF Disk Scheduling =====
Step   From     To       Movement
-----------------------------------
  1     53      65       12
  2     65      67       2
  3     67      37       30
  4     37      14       23
  5     14      98       84
  6     98      122      24
  7     122     124      2
  8     124     183      59
-----------------------------------
Sequence      : [53, 65, 67, 37, 14, 98, 122, 124, 183]
Total Seek Time: 236
ubuntu@ubuntu:~$
```

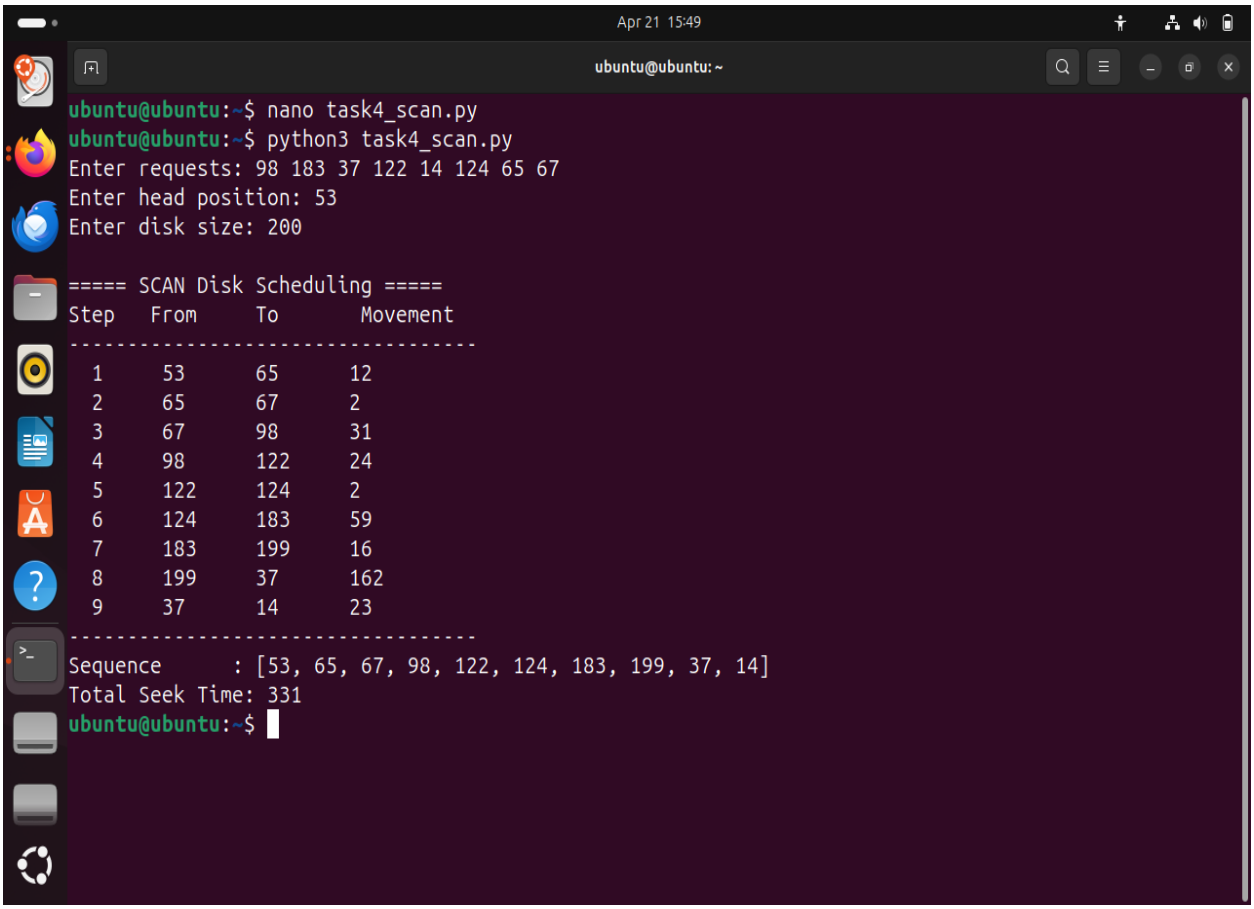
Task 4: SCAN (Elevator Algorithm)

Code :-

```
requests = list(map(int, input("Enter requests: ").split()))
head = int(input("Enter head position: "))
disk_size = int(input("Enter disk size: "))

seek_time = 0
left  = sorted([r for r in requests if r < head], reverse=True)
right = sorted([r for r in requests if r >= head])

print("\n===== SCAN =====")
for r in right:
    move = abs(head - r)
    print(f"{head} -> {r} = {move}")
    seek_time += move
    head = r
move = abs(head - (disk_size - 1))
print(f"{head} -> {disk_size-1} = {move}")
seek_time += move
head = disk_size - 1
for r in left:
    move = abs(head - r)
    print(f"{head} -> {r} = {move}")
    seek_time += move
    head = r
```

print(f"Total Seek Time: {seek_time}")

Output:-



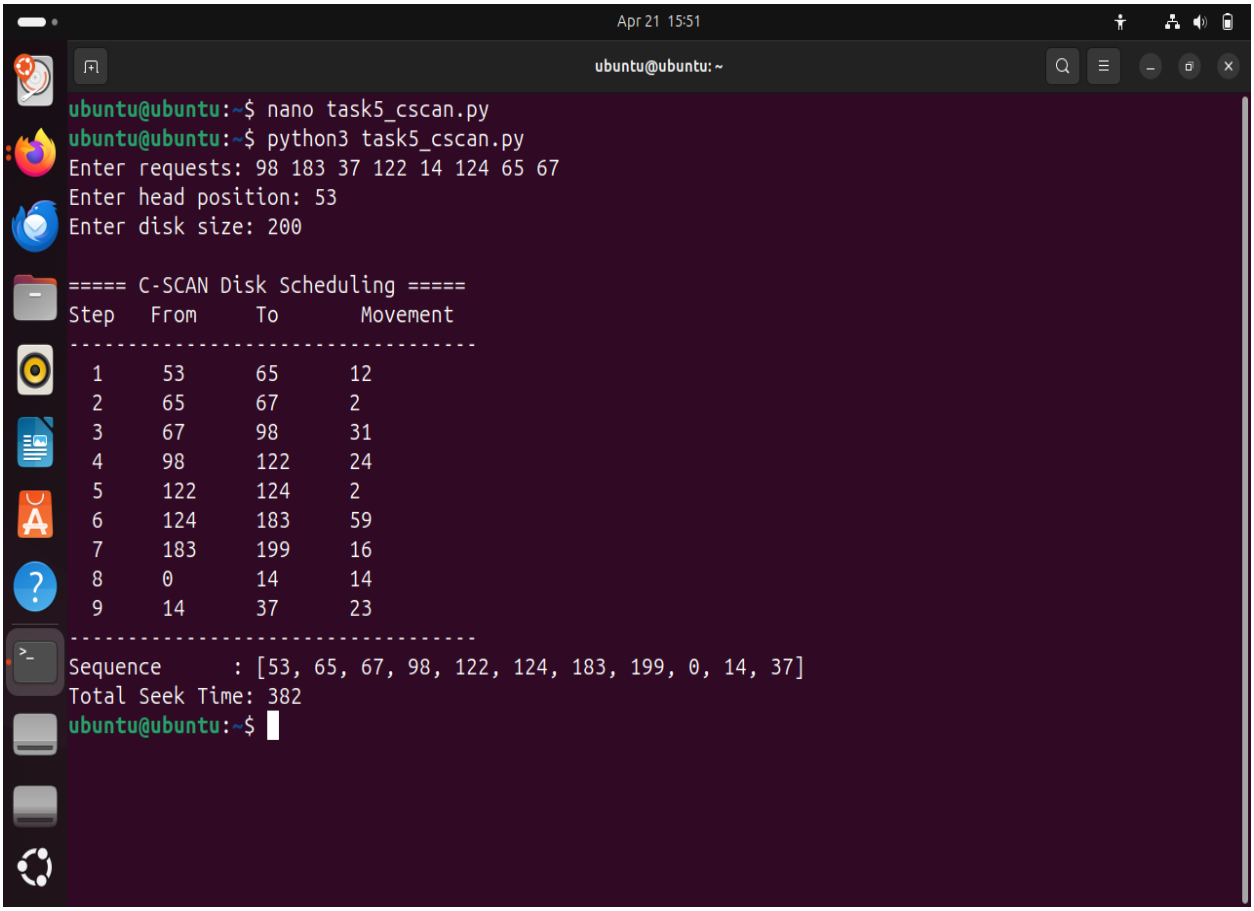
Task 5: C-SCAN Algorithm

Code :-

```
requests = list(map(int, input("Enter requests: ").split()))
head = int(input("Enter head position: "))
disk_size = int(input("Enter disk size: "))

seek_time = 0
left  = sorted([r for r in requests if r < head])
right = sorted([r for r in requests if r >= head])

print("\n===== C-SCAN =====")
for r in right:
    move = abs(head - r)
    print(f"{head} -> {r} = {move}")
    seek_time += move
    head = r
move = abs(head - (disk_size - 1))
print(f"{head} -> {disk_size-1} = {move}")
seek_time += move
seek_time += disk_size - 1
head = 0
print(f"Jump to 0")
for r in left:
    move = abs(head - r)
    print(f"{head} -> {r} = {move}")
```

```
    seek_time += move
    head = r
print(f"Total Seek Time: {seek_time}")
```

Output:-

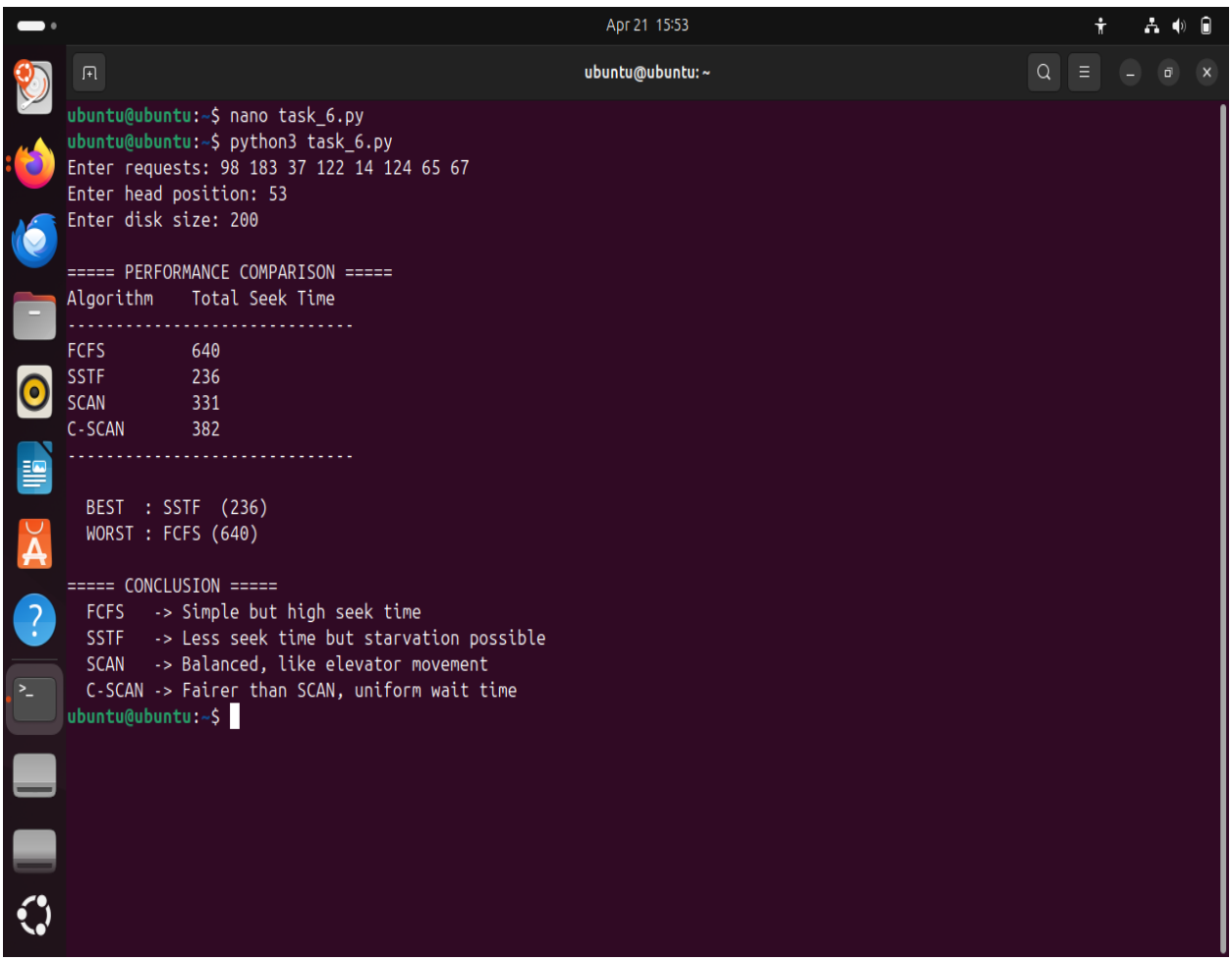


Task 6&7: Performance Comparison &Result

Code :-

```
def fcfs(req, head):
    s = 0
    for r in req:
        s += abs(head - r); head = r
    return s

def sstf(req, head):
    s, rem = 0, req.copy()
    while rem:
        n = min(rem, key=lambda x: abs(x-head))
        s += abs(head-n); head = n; rem.remove(n)
    return s

def scan(req, head, disk_size):
    s = 0
    left  = sorted([r for r in req if r < head], reverse=True)
    right = sorted([r for r in req if r >= head])
    for r in right: s += abs(head-r); head = r
    s += abs(head-(disk_size-1)); head = disk_size-1
    for r in left:  s += abs(head-r); head = r
    return s

def cscan(req, head, disk_size):
```

```
    s = 0
    left  = sorted([r for r in req if r < head])
    right = sorted([r for r in req if r >= head])
    for r in right: s += abs(head-r); head = r
    s += abs(head-(disk_size-1))
    s += disk_size-1; head = 0
    for r in left:  s += abs(head-r); head = r
    return s

requests  = list(map(int, input("Enter requests: ").split()))
head      = int(input("Enter head position: "))
disk_size = int(input("Enter disk size: "))

results = {
    'FCFS'  : fcfs(requests, head),
    'SSTF'  : sstf(requests, head),
    'SCAN'  : scan(requests, head, disk_size),
    'C-SCAN': cscan(requests, head, disk_size)
}
print("\n===== COMPARISON =====")
print(f"{'Algorithm':<10} {'Seek Time'}")
print("-" * 25)
for algo, seek in results.items():
    print(f"{algo:<10} {seek}")
best  = min(results, key=results.get)
```

```
worst = max(results, key=results.get)
print(f"\nBEST  : {best} ({results[best]})")
print(f"WORST : {worst} ({results[worst]})")
print("\n===== CONCLUSION =====")
print("FCFS   -> Simple but high seek time")
print("SSTF   -> Low seek time but starvation possible")
print("SCAN   -> Balanced like elevator")
print("C-SCAN -> Fairer, uniform wait time")
```

Output:-